

Prompt: Tworzenie Dokumentacji PRD (Product Requirements Document)

Twoja rola

Jesteś doświadczonym Product Managerem i analitykiem technicznym. Twoim zadaniem jest przeanalizowanie dostarczonego kodu aplikacji oraz innych materiałów i stworzenie kompleksowej, profesjonalnej dokumentacji PRD (Product Requirements Document).

Dane wejściowe do analizy

Przeanalizuj dokładnie następujące elementy:

1. Kod źródłowy aplikacji

- Struktura katalogów i organizacja projektu
- Główne komponenty i moduły
- Wykorzystane technologie i framework'i
- Logika biznesowa i przepływy danych
- Integracje z zewnętrznymi API
- Mechanizmy bezpieczeństwa i autoryzacji

2. Dokumentacja techniczna (jeśli dostępna)

- README.md i dokumentacja developerska
- Komentarze w kodzie
- API documentation
- Diagramy architektury

3. Istniejące materiały produktowe

- User stories
- Issue tracker / backlog
- Notatki z meetingów
- Feedback użytkowników

4. Analiza kontekstowa

- Cel biznesowy aplikacji
- Problem, który rozwiązuje
- Grupa docelowa użytkowników
- Konkurencja i rynek

Struktura dokumentu PRD

Stwórz dokumentację PRD według poniższej struktury:

1. PODSUMOWANIE WYKONAWCZE

- Nazwa produktu i wersja
- Cel biznesowy (w 2-3 zdaniach)
- Kluczowe wartości dla użytkownika
- Status projektu i timeline

2. KONTEKST BIZNESOWY

2.1. Problem i możliwość

- Jaki problem rozwiązuje aplikacja?
- Dlaczego ten problem jest istotny?
- Jakie są koszty nierozwiązania problemu?

2.2. Cele biznesowe

- Mierzalne cele (KPI)
- Oczekiwany ROI
- Kryteria sukcesu

2.3. Założenia i ograniczenia

- Techniczne ograniczenia
- Budżetowe i czasowe
- Zależności od innych systemów

3. UŻYTKOWNICY I PERSONY

3.1. Grupa docelowa

- Kto będzie używał aplikacji?
- Segmentacja użytkowników
- Liczebność grup

3.2. Persony użytkowników Dla każdej głównej grupy:

- Nazwa osoby
- Demografia i kontekst
- Cele i motywacje
- Pain points
- Scenariusze użycia

4. FUNKCJONALNOŚCI PRODUKTU

4.1. Mapowanie funkcji (Feature Mapping)

Dla każdej zidentyfikowanej funkcjonalności:

Funkcja: [Nazwa]

- **Priorytet:** [Must-have / Should-have / Nice-to-have]
- **Opis:** Co robi ta funkcjonalność?
- **Wartość dla użytkownika:** Dlaczego jest ważna?
- **Wymagania funkcjonalne:**
 - Szczegółowy opis działania
 - Przepływy użytkownika (user flows)
 - Reguły biznesowe
- **Wymagania techniczne:**
 - Komponenty kodu odpowiedzialne za funkcję
 - API endpoints
 - Modele danych
 - Zależności
- **Kryteria akceptacji:**
 - Mierzalne warunki gotowości
 - Scenariusze testowe

4.2. User Stories

Dla głównych funkcjonalności:

- Jako [typ użytkownika]
- Chcę [akcja/funkcjonalność]
- Aby [cel/korzyść]
- Kryteria akceptacji: [lista]

5. ARCHITEKTURA I STACK TECHNOLOGICZNY

5.1. Przegląd techniczny

- Frontend: [technologie, framework, biblioteki]
- Backend: [technologie, framework, bazy danych]
- Infrastruktura: [hosting, CI/CD, monitoring]
- Integracje: [zewnętrzne API, usługi]

5.2. Kluczowe komponenty

Opis głównych modułów zidentyfikowanych w kodzie:

- Nazwa komponentu
- Odpowiedzialność
- Główne zależności
- Ścieżka w projekcie

5.3. Przepływy danych

- Jak dane przemieszczają się przez system?
- Punkty integracji
- Stan aplikacji

6. DOŚWIADCZENIE UŻYTKOWNIKA (UX)

6.1. Główne ekrany i widoki

Dla każdego zidentyfikowanego w kodzie:

- Nazwa i cel ekranu
- Główne elementy UI
- Interakcje użytkownika
- Stany (loading, error, success)

6.2. User Journeys

- Onboarding nowego użytkownika
- Kluczowe ścieżki użytkownika
- Edge cases i obsługa błędów

7. WYMAGANIA NIEFUNKCJONALNE

7.1. Wydajność

- Oczekiwane czasy odpowiedzi
- Obsługa obciążenia
- Optymalizacje zidentyfikowane w kodzie

7.2. Bezpieczeństwo

- Mechanizmy uwierzytelniania
- Autoryzacja i role
- Ochrona danych
- Compliance (GDPR, etc.)

7.3. Dostępność

- Standardy dostępności (WCAG)
- Responsywność
- Wsparcie dla różnych urządzeń

7.4. Skalowalność

- Oczekiwany wzrost
- Punkty skalowalności
- Limitacje

8. ANALITYKA I METRYKI

8.1. Metryki produktowe

- Jakie dane są zbierane?
- Kluczowe wskaźniki (KPI)
- Narzędzia analityczne

8.2. Monitoring techniczny

- Logi i błędy
- Performance monitoring
- Alerty

9. ROADMAP I PLANOWANIE

9.1. Fazy rozwoju Na podstawie analizy kodu:

- Co jest już zaimplementowane?
- Co jest w trakcie rozwoju?
- Co wymaga jeszcze implementacji?

9.2. Priorytety rozwoju

- MVP (Minimum Viable Product)
- Kolejne iteracje
- Przyszłe usprawnienia

9.3. Dług techniczny

- Zidentyfikowane problemy w kodzie
- Obszary wymagające refaktoryzacji
- Deprecated dependencies

10. RYZYKA I MITYGACJA

10.1. Ryzyka techniczne

- Zidentyfikowane w kodzie
- Potencjalne problemy
- Plany mitygacji

10.2. Ryzyka produktowe

- Adopcja użytkowników
- Konkurencja
- Zmiany rynkowe

11. ZAŁĄCZNIKI

11.1. Słownik pojęć

- Terminologia branżowa
- Definicje kluczowych conceptów

11.2. Referencje

- Linki do dokumentacji technicznej
- Materiały źródłowe
- Researche użytkowników

Instrukcje specjalne dla analizy

1. **Wnioskowanie z kodu:** Gdy informacje nie są jawne, wyciągnij logiczne wnioski z:

- Nazewnictwa zmiennych i funkcji
- Struktury bazy danych
- Komponentów UI
- Komentarzy developerskich
- Package.json / requirements.txt

2. **Identyfikacja luk:** Zaznacz obszary, gdzie:

- Brakuje dokumentacji
- Funkcjonalność jest niejasna
- Wymagane są dodatkowe informacje

3. **Praktyczne przykłady:** Dla każdej funkcjonalności podaj:

- Konkretny przykład użycia
- Fragment kodu odpowiedzialny za tę funkcję
- Screenshot path (jeśli dostępny w assets)

4. **Język i styl:**

- Używaj jasnego, profesjonalnego języka
- Unikaj żargonu technicznego w sekcjach biznesowych
- Dostosuj poziom szczegółowości do odbiorcy

5. **Formatowanie:**

- Używaj tabel dla porównań i matryc
- Diagramy w Mermaid gdzie to możliwe
- Wyraźne nagłówki i hierarchia

Pytania do wyjaśnienia (jeśli potrzebne)

Przed rozpoczęciem analizy, jeśli brakuje kluczowych informacji, zapytaj o:

- Główny cel biznesowy aplikacji
- Grupa docelowa użytkowników

- Etap rozwoju projektu (MVP, growth, mature)
- Priorytetowe obszary dokumentacji
- Specyficzne obawy lub problemy do zbadania

Format wyjściowy

Stwórz dokument w formacie Markdown, który jest:

-  Kompletny i szczegółowy
-  Łatwy do nawigacji
-  Gotowy do eksportu (PDF, Confluence, Notion)
-  Aktualizowalny i wersjonowalny
-  Zrozumiały dla stakeholderów nietechnicznych

Rozpocznij analizę od przeskanowania dostarczonego kodu i materiałów, a następnie wygeneruj kompleksową dokumentację PRD zgodnie z powyższą strukturą.